

Enterprise RAG Pipeline: Technical Interview Guide

1. Project Overview

This project is an **Enterprise Retrieval-Augmented Generation (RAG) Pipeline**. The goal of a RAG system is to allow users to interact with and ask questions about private, proprietary documents without needing to fine-tune or train an AI model from scratch.

When a user uploads a PDF, the system extracts the text, mathematically encodes the meaning of that text into "vectors," and stores it in a database. When the user asks a question, the system searches the database for the most relevant pieces of text, and provides them to a Large Language Model (LLM) to formulate a highly accurate, context-aware answer.

2. Technology Stack

I engineered this system using a modern, scalable, decoupled architecture:

- **Backend Framework:** C# .NET Core API
- **Frontend Framework:** React (Vite, TypeScript, Tailwind CSS)
- **Vector Database:** MongoDB Atlas Vector Search
- **Embedding Model:** HuggingFace Free Inference API (sentence-transformers/all-MiniLM-L6-v2)
- **LLM Inference:** Groq API (llama-3.1-8b-instant model for ultra-fast generation)
- **PDF Processing:** UglyToad.PdfPig (C# library for text extraction)

3. The Architecture & Data Flow

Phase A: Data Ingestion Pipeline (The /upload Endpoint)

1. **Extraction:** The user uploads a PDF via the React UI. The backend receives the file as an `IFormFile` and passes it to the `PdfParserService`, which extracts raw text.
2. **Chunking:** The raw text is passed to the `TokenSizeChunkingService`. Large documents must be broken into smaller "chunks" (paragraphs) because LLMs have token limits, and vector databases perform better on smaller, dense text blocks.
3. **Embedding:** Each chunk is sent to the HuggingFace API. The `all-MiniLM-L6-v2` model converts the text into a **384-dimensional mathematical vector** (a list of 384 floating-point numbers representing the semantic meaning of the text).
4. **Deduplication & Storage:** To ensure the system is idempotent (safe to retry), the system first deletes any existing chunks in MongoDB that match the uploaded filename. It then performs a bulk insert (`InsertManyAsync`) to store the text chunks and their embeddings into the `DocumentChunks` collection.

Phase B: Query & Generation Pipeline (The /search Endpoint)

1. **Question Embedding:** When the user types a question in the UI, that exact question is sent to HuggingFace to be converted into a 384-dimensional vector.
2. **Vector Search:** The system executes a `$vectorSearch` aggregation against MongoDB Atlas. By comparing the cosine similarity of the question vector against all chunk vectors, MongoDB instantly returns the top 3 most "semantically similar" chunks.
 - *Note: I implemented a dynamic filter here to ensure the search only returns chunks from the currently active document (SourceFile).*
3. **LLM Generation:** The top 3 chunks are concatenated into a System Prompt: *"You are an enterprise AI assistant. Answer using strictly the provided context..."* The prompt and the user's question are sent to the Groq API.
4. **Response:** Groq's Llama 3.1 model generates the answer in milliseconds and returns it to the React UI along with the exact source file names and chunk indexes used to construct the answer.

4. Key Technical Challenges & Solutions

If an interviewer asks you about challenges you faced, you can discuss these exact scenarios:

Challenge 1: The Embedding Model Dilemma

- **Problem:** We originally designed the `GroqEmbeddingService` to use the official OpenAI SDK pointing at Groq's endpoints. However, we discovered that Groq focuses exclusively on LLM text generation and *does not host embedding models*. This caused pipeline failures (`model_not_found`).
- **Solution:** I dynamically refactored the architecture. I created a custom `HuggingFaceEmbeddingService` using a raw `HttpClient` to hit HuggingFace's free Inference API, completely bypassing the OpenAI SDK for embeddings, while retaining Groq strictly for text generation.

Challenge 2: Vector Dimension Mismatch

- **Problem:** We switched our embedding model to `all-MiniLM-L6-v2`, which outputs vectors of 384 dimensions. Our MongoDB Vector Search Index was originally configured for a different dimension size.
- **Solution:** I diagnosed the dimension mismatch and explicitly re-configured the JSON index definition in MongoDB Atlas to map to `numDimensions: 384` using `cosine` similarity.

Challenge 3: Document Duplication & Noise

- **Problem:** During testing, uploading the same PDF multiple times resulted in duplicate chunks in the database. Furthermore, searching returned chunks from *all* previously uploaded documents, confusing the LLM.
- **Solution:** I implemented a two-part fix. First, I added pre-ingestion deletion logic (`DeleteManyAsync`) to ensure idempotent uploads. Second, I introduced state-tracking in the React UI (`activeDocument`) and passed it to the backend to inject a strict `$vectorSearch` filter, effectively isolating the chat context to the active document.

Challenge 4: Model Deprecation Mid-Development

- **Problem:** Groq decommissioned the `llama3-8b-8192` model without warning, throwing `HTTP 400 (invalid_request_error: model_decommissioned)` during testing.
- **Solution:** Analyzed the stack trace, identified the deprecated hardcoded string, and upgraded the system to use their state-of-the-art replacement: `llama-3.1-8b-instant`.

Challenge 5: Security & Secret Management

- **Problem:** While pushing the code to GitHub, the push was rejected because GitHub's secret-scanning detected our live Groq and HuggingFace API tokens in `appsettings.json`.
 - **Solution:** I intercepted the commit, sanitized the `appsettings.json` with placeholder strings, amended the git commit to permanently erase the history of the keys, successfully pushed the safe code, and then restored the local keys so the development environment remained functional.
-

5. UI/UX Engineering

Instead of a generic frontend, I engineered a highly premium interface to showcase the capabilities of the backend:

- **Dark Glassmorphism:** Utilized Tailwind CSS to create translucent, frosted-glass panels (`backdrop-blur-xl`) resting over a dark `slate-950` background.
- **Micro-animations:** Built custom CSS keyframe animations for floating background orbs to make the AI feel dynamic and "alive."
- **Feedback Loops:** Implemented intelligent UI states (e.g., animated bouncing dots for the "Analyzing knowledge base..." phase, and dynamic upload spinners) to keep the user informed during asynchronous backend operations.